QUANTUM COMPUTING

Assignment 2

Simon's Algorithm: Implementation and Analysis

Harshey Sharma

GitHub: https://github.com/Harshey260/harshey_quantum2_assignment.git

Part A — Understanding the Reference Implementation

The reference implementation demonstrates Simon's Algorithm for the secret string `s = 011`. The algorithm uses `n = 3` qubits for the query register and n = 3 qubits for the ancilla register, giving 6 qubits total, with 3 classical bits for measurement.

Role of Each Component

1. First Hadamard Layer

Applied to all n qubits in the query register at the start, the Hadamard gates transform each qubit from $|0\rangle$ into an equal superposition of $|0\rangle$ and $|1\rangle$. Collectively, this creates a uniform superposition of all 2^n possible n-bit input states simultaneously:

$$|0\rangle^{\otimes n} \rightarrow (1/\sqrt{2^n}) \sum |x\rangle \quad (x = 0 \ldots 2^n-1)$$

This allows the oracle to be evaluated on all inputs at once — the key quantum advantage. A classical computer would need to evaluate f(x) one input at a time.

2. The Oracle

The oracle encodes the secret string *s* by implementing the 2-to-1 function $f(x) = f(x \oplus s)$, where each function value is shared by exactly two inputs.

The oracle is built in two stages:

- Copy step: CNOT gates copy the query register to the ancilla register — creating f(x) = x for all x.
- Collision step: For the index m of the first '1' in the reversed secret string, additional CNOT gates are applied from qubit m to all ancilla qubits at '1' positions in s. This XORs in the secret pattern and enforces $f(x) = f(x \oplus s)$.

For s = 011 (reversed: 110), m = 0. CNOT gates are applied from qubit 0 to ancilla qubits 3 and 4 — the positions of the two '1' bits in the reversed string.

3. Second Hadamard Layer

After the oracle, a second layer of Hadamard gates is applied to the query register, performing quantum interference:

- States that do NOT satisfy $u \cdot s = 0 \pmod 2$ destructively cancel out.
- States that DO satisfy this equation constructively reinforce.

The amplitude of any output state $|u\rangle$ is proportional to:

$$\sum (-1)^{(u \cdot x)} |f(x)\rangle \rightarrow \text{non-zero only when } u \cdot s = 0 \pmod 2$$

4. Measurement

Only the query register (qubits 0 to n−1) is measured. Each shot produces a bit string u. Because of the interference created by the second Hadamard layer, only strings satisfying $u \cdot s = 0 \pmod 2$ are ever observed — all others have zero probability amplitude.

Repeating the circuit O(n) times yields enough independent equations to reconstruct s classically.

Why Measurement Outputs Satisfy $u \cdot s = 0 \pmod 2$

After the oracle entangles the query and ancilla registers, the full state is:

$$|\psi\rangle = (1/\sqrt{2^n}) \sum_x |x\rangle |f(x)\rangle$$

Since f satisfies the Simon promise, for each function value v, exactly two inputs x and $x \oplus s$ map to it. After applying the second Hadamard and projecting onto ancilla state $|v\rangle$, the amplitude of $|u\rangle$ in the query register is proportional to:

$$(-1)^{(u \cdot x)} [1 + (-1)^{(u \cdot s)}]$$

This amplitude is zero unless $(-1)^{(u \cdot s)} = 1$, i.e., $u \cdot s = 0 \pmod 2$. Therefore only strings u orthogonal to s (mod 2) can ever be observed.

Verification for s = 011

For s = 011, the constraint $u \cdot s = 0 \pmod 2$ becomes: $u[1] \oplus u[0] = 0$ — the two rightmost bits must be equal. This yields exactly 4 valid strings:

Bit String u	Valid? ($u[1] \oplus u[0] = 0$)
000	✓ Yes
011	✓ Yes
100	✓ Yes
111	✓ Yes

SIM Simulation output (2000 shots): {'011': 518, '000': 492, '111': 499, '100': 491} — only these 4 strings appeared, confirming the algorithm is correct.

Part B — Simon's Algorithm for s = 10110

The algorithm is implemented for the 5-bit secret string `s = 10110`. With n = 5, the circuit requires 10 qubits (5 query + 5 ancilla) and 5 classical bits.

Oracle Construction for s = 10110

The oracle is built using the same scalable logic as Part A, accounting for Qiskit's little-endian qubit ordering:

- Copy step: cx(i, i+5) for i = 0…4 — copies query register to ancilla.
- Collision: cx(1,6), cx(1,7), cx(1,9) — for the three '1' positions in s_rev at indices 1, 2, and 4.

Circuit Execution

The 10-qubit circuit was executed using the Qiskit Aer simulator with 2000 shots. The expected number of valid measurement outcomes is `2^(n-1) = 2^4 = 16` (including the trivial 00000).

Measured Bit Strings (excluding trivial 00000)

The following 15 non-trivial bit strings u were obtained. All satisfy u · s = 0 (mod 2):

u_1	u_2	u_3	u_4	u_5	u_6
00001	01000	01001	00110	00111	01110
01111	10010	10011	11010	11011	10100
10101	11100	11101	—	—	—

✓ All 16 expected valid strings (including 00000) appeared in the simulation, confirming the oracle is correctly implemented.

Part C — Classical Post-Processing

Each measured bit string u satisfies the inner product equation `u · s = 0 (mod 2)`. Collectively these form the matrix system `Us = 0 (mod 2)`, where U is the matrix whose rows are the measured u strings.

System of Linear Equations

Writing $s = (s_4, s_3, s_2, s_1, s_0)$, each measured string gives one equation. Key independent equations:

u string	Equation (u · s = 0 mod 2)	Simplified
00001	$0 \cdot s_4 + 0 \cdot s_3 + 0 \cdot s_2 + 0 \cdot s_1 + 1 \cdot s_0 = 0$	$s_0 = 0$
01000	$0 \cdot s_4 + 1 \cdot s_3 + 0 \cdot s_2 + 0 \cdot s_1 + 0 \cdot s_0 = 0$	$s_3 = 0$
01001	$0 \cdot s_4 + 1 \cdot s_3 + 0 \cdot s_2 + 0 \cdot s_1 + 1 \cdot s_0 = 0$	$s_3 + s_0 = 0$
00110	$0 \cdot s_4 + 0 \cdot s_3 + 1 \cdot s_2 + 1 \cdot s_1 + 0 \cdot s_0 = 0$	$s_2 + s_1 = 0$
00111	$0 \cdot s_4 + 0 \cdot s_3 + 1 \cdot s_2 + 1 \cdot s_1 + 1 \cdot s_0 = 0$	$s_2 + s_1 + s_0 = 0$
10010	$1 \cdot s_4 + 0 \cdot s_3 + 0 \cdot s_2 + 1 \cdot s_1 + 0 \cdot s_0 = 0$	$s_4 + s_1 = 0$
10100	$1 \cdot s_4 + 0 \cdot s_3 + 1 \cdot s_2 + 0 \cdot s_1 + 0 \cdot s_0 = 0$	$s_4 + s_2 = 0$
11100	$1 \cdot s_4 + 1 \cdot s_3 + 1 \cdot s_2 + 0 \cdot s_1 + 0 \cdot s_0 = 0$	$s_4 + s_3 + s_2 = 0$
11010	$1 \cdot s_4 + 1 \cdot s_3 + 0 \cdot s_2 + 1 \cdot s_1 + 0 \cdot s_0 = 0$	$s_4 + s_3 + s_1 = 0$
11101	$1 \cdot s_4 + 1 \cdot s_3 + 1 \cdot s_2 + 0 \cdot s_1 + 1 \cdot s_0 = 0$	$s_4 + s_3 + s_2 + s_0 = 0$

(The remaining 5 equations are linearly dependent on these and are omitted for brevity — all 15 were used in the solver.)

Solver Code (Python)

A brute-force Python solver tests all 31 non-zero 5-bit candidates against the full set of equations:

```
n = 5
valid_u_strings = ['00001','01000','01001','00110','00111',
                   '01110','01111','10010','10011','11010',
                   '11011','10100','10101','11100','11101']

for i in range(1, 2**n):
    candidate = format(i, f'0{n}b')
    valid = all(
        sum(int(u[j])*int(candidate[j]) for j in range(n)) % 2 == 0
        for u in valid_u_strings
    )
    if valid:
        print('Recovered s:', candidate)
```

Results Summary

Parameter	Value
Original secret string (s)	10110
Recovered secret string	10110
Match	✓ Perfect match
Equations used	15 (all non-trivial u strings)
Expected valid strings	$2^4 = 16$
Observed valid strings	16 (including 00000)

Conclusion

The classical post-processing successfully and uniquely recovered s = 10110 from the quantum measurement results. The uniqueness of the solution demonstrates that a sufficient number of linearly independent equations were collected — for an n-bit secret string, only $n-1 = 4$ linearly independent equations are needed, and the 15 equations obtained from 2000 shots easily satisfy this requirement.

KEY This assignment demonstrates the full Simon's algorithm pipeline: quantum superposition → oracle evaluation → quantum interference → classical linear algebra, achieving exponential speedup over any classical approach.